

Tutorial/Lab 3 – Abstract Classes and Interfaces

Aim

This tutorial/lab aims to further the understanding of the topics covered in Week 3's lecture, such as abstract classes, interfaces and deep copy.

Exercises

Exercise 3.1: ArrayList Shuffling

Write the following method that shuffles an `ArrayList` of numbers:

```
public static void shuffle(ArrayList<Number> list)
```

Hints:

1. Create the `ArrayList` first
2. In the `shuffle()` method, you may consider the Fisher-Yates shuffle algorithm, which is efficient for shuffling.
3. Generate a random index within the range of the list size using `Math.random()`.
4. Swap the element at index `i` with the element at the randomly generated index to perform the shuffle.

Exercise 3.2: Comparable Interface

Define a class named `ComparableCircle` that extends `Circle` (you can download the class file `Circle.java` and its superclass `GeometricObject.java` on Learning Mall) and implements `Comparable`. Implement the `compareTo` method to compare the circles on the basis of radius. Write a test class to find the larger of two instances of `ComparableCircle` objects.

Hints:

1. Define a class named `ComparableCircle` that extends `Circle` and implements `Comparable` interface. You need to implement the `compareTo` method to compare the circles based on their radii.
2. Implement the `compareTo` method in the `ComparableCircle` class to compare circles based on their radii. You should compare the radii of two circles and return a positive integer if the radius of the current circle is greater, a negative integer if it's smaller, and zero if they are equal.

3. Write a test class to verify the functionality of the `ComparableCircle` class. Create two `ComparableCircle` objects and compare their sizes using the `Max.max` method (in `Max.java` – you can download this file on Learning Mall).

Exercise 3.3: Deep Copy

Consider the `MyStack` class from Lecture 2.2 (you can also download the class file `MyStack.java` from Learning Mall):

```
1 public class MyStack {
2     private java.util.ArrayList list = new java.util.ArrayList();
3     public void push(Object o) {
4         list.add(o);
5     }
6     public Object pop() {
7         Object o = list.get(getSize() - 1);
8         list.remove(getSize() - 1);
9         return o;
10    }
11    public Object peek() {
12        return list.get(getSize() - 1);
13    }
14    public int search(Object o) {
15        return list.lastIndexOf(o);
16    }
17    public boolean isEmpty() {
18        return list.isEmpty();
19    }
20    public int getSize() {
21        return list.size();
22    }
23    public String toString() {
24        return "stack: " + list.toString();
25    }
26 }
27
```

Rewrite this class to perform a deep copy of the list field.

Hints:

1. Implement the `Cloneable` interface in the `MyStack` class to enable cloning functionality.
2. Override the clone method in the `MyStack` class to create a deep copy of the list field.
3. Within the clone method, clone each object in the list to ensure a deep copy is made.

Exercise 3.4: Abstract Superclass for Employees

Define an abstract class named `Employee` with the following:

- Fields: `name` and `id`
- A constructor to initialize these fields
- Getters for both fields
- An abstract method `double calculateSalary()`
- A `toString()` method that returns the employee's basic information

Then create two subclasses:

- `FullTimeEmployee`, with an additional field `monthlySalary`
- `PartTimeEmployee`, with additional fields `hoursWorked` and `hourlyRate`

Requirements:

- Override `calculateSalary()` appropriately in each subclass.
- Write a test class that creates one object of each subclass.
- Store them in an `Employee[]` array.
- Use a loop to print each employee's information and salary.

Exercise 3.5: Interface for Discountable Products

Define an interface named `Discountable` with one method:

```
double getDiscountedPrice(double percent);
```

Then create a class named `Product` with fields: `name` and `price`

Create a subclass named `Book` that extends `Product` and implements `Discountable`.

Requirements:

- Implement the `getDiscountedPrice(double percent)` method in `Book`.
- Add a `toString()` method to display the book's name and original price.
- Write a test class that creates a `Book` object and prints:
 - the original price
 - the discounted price after applying a given percentage

Hint: The discounted price can be calculated as: `price * (1 - percent / 100.0)`

Exercise 3.6: Abstract Class vs. Interface

1. Consider the following code snippet. Will it compile successfully? If not, what is the reason?

```
1 public interface Animal {  
2     String name; // Data field representing the name of the animal  
3     void makeSound(); // Abstract method to make the animal sound  
4 }
```

2. Consider the following code snippet. Will it compile successfully? If not, what is the reason?

```
1 public interface MyInterface2 {  
2     void method1(); // Abstract method without implementation  
3     void method2(); // Abstract method without implementation  
4     void method3() {  
5         // Concrete method with implementation  
6         System.out.println("Method 3 implementation");  
7     }  
8 }  
9
```

3. What is the issue in the following code snippet? How should it be solved?

```
1 public abstract class MyClass {  
2     public MyClass() {  
3         System.out.println("Abstrac class constructor");  
4     }  
5  
6     public static void main(String[] args) {  
7         MyClass obj = new MyClass();  
8     }  
9 }  
10
```

Exercise 3.7: Comparable Interface for Students

Define a class named `Student` that implements `Comparable<Student>` and contains the following fields: `name` and `mark`.

Requirements:

- Write a constructor to initialize both fields.
- Implement the `compareTo` method so that students are compared by `mark`.
- If two students have the same mark, compare them by `name` alphabetically.
- Write a test class that creates an array of `Student` objects.
- Sort the array using `Arrays.sort(...)`.
- Print the sorted result.

Exercise 3.8: Using the Number Class

Write a program that creates an array of `Number` references containing a mixture of `Integer` and `Double`.

Requirements:

- Store at least five values in a `Number[]` array.
- Use a loop to calculate the sum of all values in the array.
- Print each value and its type.
- Print the final sum as a `double`.

Hints:

- Use wrapper objects such as `Integer.valueOf(...)` and `Double.valueOf(...)`, or rely on autoboxing.
- Since all elements are of type `Number`, you can use methods such as `doubleValue()`.
- To print the type, you may use `getClass().getSimpleName()`.